OSSP - Skill Week 2 Report

Name: Tejaswin Amara

Roll No: 2520090104

Department: CS&IT

Course: OSSP

Week: 2

Objective

To install and configure the Linux development environment, create the required OSSP project structure, initialize a Git repository, and understand process creation using the fork() system call.

Task 1: Create OSSP Project Structure

Commands Executed:1. Created the OSSP project directory.2. Created the required folders using mkdir.3. Created source, header, documentation, test, and helper files using touch.4. Initialized a Git repository using git init.5. Verified the project structure using ls -R.

Screenshots

```
speed@DESKTOP-P4G181S:~$ cd ~
speed@DESKTOP-P4G181S:~$ mkdir OSSP
speed@DESKTOP-P4G181S:~$ cd OSSP
speed@DESKTOP-P4G181S:~/OSSP$ mkdir src include obj bin docs tests scripts assets
speed@DESKTOP-P4G181S:~/OSSP$ touch .gitignore README.md LICENSE Makefile
touch src/main.c src/shell.c src/parser.c src/executor.c src/builtin.c
touch include/shell.h include/parser.h include/executor.h include/builtin.h
touch docs/design.md docs/report.pdf
touch tests/test_parser.c tests/test_executor.c
touch scripts/run.sh
touch assets/architecture.png
speed@DESKTOP-P4G181S:~/OSSP$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: will change to "main" in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
```

```
Initialized empty Git repository in /home/speed/OSSP/.git/
speed@DESKTOP-P4G181S:~/OSSP$ tree OSSP
Command 'tree' not found, but can be installed with:
sudo snap install tree  # version 2.1.3+pkg-5852, or
sudo apt  install tree  # version 2.3.1-1
See 'snap info tree' for additional versions.
speed@DESKTOP-P4G181S:~/OSSP$ sudo apt install tree
[sudo: authenticate] Password:
Installing:
  tree

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 13
  Download size: 53.5 kB
  Space needed: 125 kB / 1024 GB available

Get:1 http://archive.ubuntu.com/ubuntu resolute/universe amd64 tree amd64 2.3.1-1 [53.5 kB]
Fetched 53.5 kB in 1s (48.1 kB/s)
Selecting previously unselected package tree.
(Reading database ... 42369 files and directories currently installed.)
Preparing to unpack .../tree_2.3.1-1_amd64.deb ...
Unpacking tree (2.3.1-1) ...
Setting up tree (2.3.1-1) ...
Processing triggers for man-db (2.13.1-1build1) ...
speed@DESKTOP-P4G181S:~/OSSP$ ls -R
.:
LICENSE   README.md  bin   include  scripts  tests
Makefile  assets     docs  obj      src

./assets:
architecture.png

./bin:

./docs:
design.md  report.pdf
```

```
speed@DESKTOP-P4G181S:~/OSSP$ ls -R
.:
LICENSE   README.md  bin   include  scripts  tests
Makefile  assets     docs  obj      src

./assets:
architecture.png

./bin:

./docs:
design.md  report.pdf

./include:
builtin.h  executor.h  parser.h  shell.h

./obj:

./scripts:
run.sh

./src:
builtin.c  executor.c  main.c  parser.c  shell.c

./tests:
test_executor.c  test_parser.c
speed@DESKTOP-P4G181S:~/OSSP$
```

Task 2: Process Creation using fork()

A C program was written to demonstrate process creation using the fork() system call. The program displays the Parent Process ID (PID) and Child Process ID (PID).

```
speed@DESKTOP-P4G181S:~$ nano fork_example.c
speed@DESKTOP-P4G181S:~$ gcc fork_example.c -o fork_example
speed@DESKTOP-P4G181S:~$ ./fork_example
This is the Parent Process.
Parent PID = 1040
Child PID = 1041
This is the Child Process.
Child PID = 1041
speed@DESKTOP-P4G181S:~$
```

```
  GNU nano 8.7.1                fork_example.c
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
    }
    else if (pid == 0) {
        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());
    }
    else {
        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }

    return 0;
}
```

Observation

The fork() system call creates a child process. Both parent and child execute independently and have different process IDs.

Conclusion

The objectives of Week 2 Skill were successfully completed. The Linux development environment was configured, the required project structure was created, Git was initialized, and process creation using fork() was demonstrated successfully.